

Data-Driven Optimization

Traveling Salesman Problem

과제 4

문제 1. TSP – book salesperson

- Basin에 살고 있는 책 영업사원은 한달에 한 번 Wald, Bon, Mena, Kiln에 있는 4명의 고객을 방문하고 있다. 다음 표는 각 도시간의 거리를 마일로 표시한 표이다. 총 거리를 최소화하는 경로를 찾아라.

	Miles between cities				
	<i>Basin</i>	<i>Wald</i>	<i>Bon</i>	<i>Mena</i>	<i>Kiln</i>
Basin	0	125	225	155	215
Wald	125	0	85	115	135
Bon	225	85	0	165	190
Mena	155	115	165	0	195
Kiln	215	135	190	195	0

- 1) 지난 주 공부한 수리 모형을 기반으로 최적해를 구하라.
- 2) 금주에 공부한 ortools code를 활용하여 최적해를 구하라.
- 3) TPS에 대한 해를 찾는 가장 간편한 방법은 방문한 도시에서 가장 가까운 도시를 선택하여 경로를 찾는 nearest neighbor algorithm이다. 이 방식으로 제시된 문제에 대한 경로를 찾고 1), 2) 해와 비교하여라.

문제 1. 답안

```
from ortools.linear_solver import pywraplp
import pandas as pd
```

```
DIST = [ [0, 125, 225, 155, 215],
         [125, 0, 85, 115, 135],
         [225, 85, 0, 165, 190],
         [155, 115, 165, 0, 195],
         [215, 135, 190, 195, 0]
       ]
```

```
nCity = len(DIST)
```

```
solver = pywraplp.Solver.CreateSolver("SAT")
```

```
X = {}
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            X[i, j] = solver.IntVar(0, 1, "X"+str(i)+str(j))
```

```
Y = {}
for i in range(nCity):
    Y[i] = solver.IntVar(0, 1, "U"+str(i))
```

```
U = {}
for i in range(1, nCity):
    U[i] = solver.NumVar(1, nCity-1, "U"+str(i))
```

```
# 제약조건
# 도시 j로 한 번은 들어와야 한다.
for j in range(nCity):
    solver.Add(solver.Sum([X[i, j] for i in range(nCity) if i != j]) == 1, 'in_'+str(i))
```

도시 j로 부터 한 번은 나와야 한다.

```
for i in range(nCity):
    solver.Add(solver.Sum([X[i, j] for j in range(nCity) if i != j]) == 1, 'out_'+str(i))
```

```
for i in range(1, nCity):
    for j in range(1, nCity):
        if i != j:
            solver.Add(U[i] - U[j] + nCity*X[i,j] - (nCity-1)<= 0, 'U_'+str(i)+str(j))
```

Objective

```
objective_terms = []
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            objective_terms.append(DIST[i][j] * X[i, j])
```

```
solver.Minimize(solver.Sum(objective_terms))
```

```
if 1:
    with open('hw4-1.lp', "w") as out_f:
        lp_text = solver.ExportModelAsLpFormat(False)
        out_f.write(lp_text)
```

Solve

```
status = solver.Solve()
```

Print solution. 답안 출력

```
if status == pywraplp.Solver.OPTIMAL or status == pywraplp.Solver.FEASIBLE:
    print(f"Total cost = {solver.Objective().Value():.1f}\n", )
    for i in range(nCity):
        for j in range(nCity):
            if i != j:
                if X[i, j].solution_value() > 0.5:
                    print(f"X{i} --> X{j}")
else:
    print("No solution found.")
```

문제 2. Open Tour

- 어떤 관광객은 뉴욕에서 8개의 장소를 방문하고자 한다. 8개 장소의 방문 순서, 시작과 끝은 상관없고 출발장소로 돌아올 필요도 없다고 하자(Open Tour). 다음은 각 장소 간의 택시비를 표현한 것이다.

$$\|c_{ij}\| = \begin{pmatrix} 0 & 20 & 30 & 25 & 12 & 33 & 44 & 57 \\ 22 & 0 & 19 & 20 & 20 & 29 & 43 & 45 \\ 28 & 19 & 0 & 17 & 38 & 48 & 55 & 60 \\ 25 & 20 & 19 & 0 & 28 & 35 & 40 & 55 \\ 12 & 18 & 34 & 25 & 0 & 21 & 30 & 40 \\ 35 & 25 & 45 & 30 & 20 & 0 & 25 & 39 \\ 47 & 39 & 50 & 35 & 28 & 20 & 0 & 28 \\ 60 & 38 & 54 & 50 & 33 & 40 & 25 & 0 \end{pmatrix}$$

- 이 관광객은 예산으로 135달러를 책정했다면 이 관광 계획은 가능한가?
- 만약 마지막 방문 장소를 6번으로 하고 싶다면 최소 얼마의 예산이 필요한가?

문제 2. Open Tour 답안

```
from ortools.linear_solver import pywraplp
import pandas as pd
```

```
DIST = [
    [0,20,30,25,12,33,44,57,0],
    [22,0,19,20,20,29,43,45,0],
    [28,19,0,17,38,48,55,60,0],
    [25,20,19,0,28,35,40,55,0],
    [12,18,34,25,0,21,30,40,0],
    [35,25,45,30,20,0,25,39,0],
    [47,39,50,35,28,20,0,28,0],
    [60,38,54,50,33,40,25,0,0],
    [0,0,0,0,0,0,0,0,0]
```

```
]
nCity = len(DIST)
```

```
solver = pywraplp.Solver.CreateSolver("SAT")
```

```
X = {}
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            X[i, j] = solver.IntVar(0, 1, "X"+str(i)+str(j))
```

```
Y = {}
for i in range(nCity):
    Y[i] = solver.IntVar(0, 1, "U"+str(i))
```

```
U = {}
for i in range(1, nCity):
    U[i] = solver.NumVar(1, nCity-1, "U"+str(i))
```

```
# 제약조건
# 도시 j로 한 번은 들어와야 한다.
for j in range(nCity):
    solver.Add(solver.Sum([X[i, j] for i in range(nCity) if i != j]) == 1, 'in_'+str(j))
```

```
# 도시 j로 부터 한 번은 나와야 한다.
for i in range(nCity):
    solver.Add(solver.Sum([X[i, j] for j in range(nCity) if i != j]) == 1, 'out_'+str(i))
```

```
for i in range(1, nCity):
    for j in range(1, nCity):
        if i != j:
            solver.Add(U[i] - U[j] + nCity*X[i,j] - (nCity-1)<= 0, 'U_'+str(i)+str(j))
```

```
#solver.Add(X[6, 8] == 1) # 6번에서 가상 방문지로 가는 링크를 1로 하면 6번이 마지막 방문지가 됨.
```

```
# Objective
objective_terms = []
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            objective_terms.append(DIST[i][j] * X[i, j])
```

```
solver.Minimize(solver.Sum(objective_terms))
```

```
# Solve
status = solver.Solve()
```

```
# 답안 출력 부분
```

문제 3. TSP – 서비스 센터

- 시어스 서비스센터는 매일 고객 방문을 스케줄 한다. 다음 표는 서비스센터와 7명의 고객 사이의 이동시간을 분으로 나타낸다. 수리 기사는 모든 고객을 방문하고 회사로 돌아와야 한다고 가정한다.

$$\|T_{ij}\| = \begin{pmatrix} 0 & 20 & 15 & 19 & 24 & 14 & 21 & 11 \\ 20 & 0 & 18 & 22 & 23 & 22 & 9 & 10 \\ 15 & 18 & 0 & 11 & 21 & 14 & 32 & 12 \\ 19 & 22 & 11 & 0 & 20 & 27 & 18 & 15 \\ 24 & 23 & 21 & 20 & 0 & 14 & 25 & 20 \\ 14 & 22 & 14 & 27 & 14 & 0 & 26 & 17 \\ 21 & 9 & 32 & 18 & 25 & 26 & 0 & 20 \\ 11 & 10 & 12 & 15 & 20 & 17 & 20 & 0 \end{pmatrix}$$

- 위의 모든 고객을 방문하는 시간을 최소화하는 방문순서를 결정하는 모델을 수립하여라.
- 고객 2, 3번은 아주 오래 기다린 고객으로 수리기사가 먼저 방문해야 하고, 2번이 사용하던 제품을 6번 고객이 사용하므로 2번 고객 방문 다음은 반드시 6번 고객이 되어야 한다. 이 내용을 반영하기 위해 모델을 수정하여라.

문제 3. 서비스 센터 답안

```
from ortools.linear_solver import pywraplp
import pandas as pd
```

```
DIST = [ [0, 20, 15, 19, 24, 14, 21, 11],
          [20, 0, 18, 22, 23, 22, 9, 10],
          [15, 18, 0, 11, 21, 14, 32, 12],
          [19, 22, 11, 0, 20, 27, 18, 15],
          [24, 23, 21, 20, 0, 14, 25, 20],
          [14, 22, 14, 27, 14, 0, 26, 17],
          [21, 9, 32, 18, 25, 26, 0, 20],
          [11, 10, 12, 15, 20, 17, 20, 0]
```

```
    ]
nCity = len(DIST)
```

```
solver = pywraplp.Solver.CreateSolver("SAT")
```

```
X = {}
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            X[i, j] = solver.IntVar(0, 1, "X"+str(i)+str(j))
```

```
Y = {}
for i in range(nCity):
    Y[i] = solver.IntVar(0, 1, "U"+str(i))
```

```
U = {}
for i in range(1, nCity):
    U[i] = solver.NumVar(1, nCity-1, "U"+str(i))
```

```
# 제약조건
# 도시 j로 한 번은 들어와야 한다.
for j in range(nCity):
    solver.Add(solver.Sum([X[i, j] for i in range(nCity) if i != j]) == 1, 'in_'+str(j))

# 도시 j로 부터 한 번은 나와야 한다.
for i in range(nCity):
    solver.Add(solver.Sum([X[i, j] for j in range(nCity) if i != j]) == 1, 'out_'+str(i))

for i in range(1, nCity):
    for j in range(1, nCity):
        if i != j:
            solver.Add(U[i] - U[j] + nCity*X[i,j] - (nCity-1)<= 0, 'U_'+str(i)+str(j))

#solver.Add(X[0,1] == 1)
solver.Add(U[1] >= U[5] )

# Objective
objective_terms = []
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            objective_terms.append(DIST[i][j] * X[i, j])

solver.Minimize(solver.Sum(objective_terms))

if 1:
    with open('hw4-1.lp', 'w') as out_f:
        lp_text = solver.ExportModelAsLpFormat(False)
        out_f.write(lp_text)

# Solve
status = solver.Solve()

# Print solution. 답안 출력
```

문제 4. TSP – 미팅 순서

- 어떤 관리자는 10명의 직원과 함께 6개의 프로젝트를 진행하고 있다. 각 직원은 2 ~ 5개의 프로젝트에 참가하고 있고 관리자는 매주 월요일 오전에 6개 프로젝트에 대한 미팅을 진행하고 있다(프로젝트당 30분, 총 3시간 미팅). 프로젝트 미팅에는 참여 직원이 모두 참여해야 한다고 가정할 때 회의실에 들어오고 나가는 직원수가 최소가 되도록 프로젝트 미팅 순서를 결정하고자 한다. 프로젝트에 참여 현황은 우측과 같다.

- 1) 프로젝트 미팅 순서를 결정하기 위한 TSP의 거리 행렬을 만들어라.
- 2) TSP 문제를 풀어 미팅 순서를 결정하여라. (역시 Open-Tour TSP)

		Project					
		1	2	3	4	5	6
Employee	1		X		X	X	
	2	X		X		X	
	3		X	X	X		X
	4			X	X	X	
	5	X	X	X			
	6	X	X	X	X		X
	7	X	X			X	X
	8	X		X	X		
	9					X	X
	10	X	X		X	X	X

문제 4. 미팅 순서 답안

```
from ortools.linear_solver import pywraplp
import pandas as pd
```

```
DIST = [
    [0, 4, 4, 6, 6, 5, 0],
    [4, 0, 6, 4, 6, 3, 0],
    [4, 6, 0, 4, 6, 6, 0],
    [6, 4, 4, 0, 6, 5, 0],
    [6, 6, 6, 6, 0, 5, 0],
    [5, 3, 6, 5, 6, 0, 0],
    [0, 0, 0, 0, 0, 0, 0]
]
```

```
nCity = len(DIST)
```

```
solver = pywraplp.Solver.CreateSolver("SAT")
```

```
X = {}
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            X[i, j] = solver.IntVar(0, 1, "X"+str(i)+str(j))
```

```
Y = {}
for i in range(nCity):
    Y[i] = solver.IntVar(0, 1, "U"+str(i))
```

```
U = {}
for i in range(1, nCity):
    U[i] = solver.NumVar(1, nCity-1, "U"+str(i))
```

```
# 제약조건
# 도시 j로 한 번은 들어와야 한다.
for j in range(nCity):
    solver.Add(solver.Sum([X[i, j] for i in range(nCity) if i != j]) == 1, 'in_'+str(j))

# 도시 j로 부터 한 번은 나와야 한다.
for i in range(nCity):
    solver.Add(solver.Sum([X[i, j] for j in range(nCity) if i != j]) == 1, 'out_'+str(i))

for i in range(1, nCity):
    for j in range(1, nCity):
        if i != j:
            solver.Add(U[i] - U[j] + nCity*X[i, j] - (nCity-1) <= 0, 'U_'+str(i)+str(j))

# Objective
objective_terms = []
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            objective_terms.append(DIST[i][j] * X[i, j])

solver.Minimize(solver.Sum(objective_terms))

if 0:
    with open('hw4-4.lp', 'w') as out_f:
        lp_text = solver.ExportModelAsLpFormat(False)
        out_f.write(lp_text)

# Solve
status = solver.Solve()

# Print solution.
```

문제 5. TSP – 식사 배달

- 어떤 자선 단체는 1명의 배달 기사를 통해 7명의 회원에게 도시락 배달을 하고자 한다. 식당은 0에 위치하고 있고 배송이후 배달 기사는 식당으로 돌아올 필요는 없다(Open tour again). 아래 표는 식당과 각 회원 간의 이동 시간을 나타낸다.

$$\|t_{ij}\| = \begin{pmatrix} 0 & 10 & 12 & 5 & 17 & 9 & 13 & 7 \\ 10 & 0 & 9 & 20 & 8 & 11 & 3 & 5 \\ 12 & 9 & 0 & 14 & 4 & 10 & 1 & 16 \\ 5 & 20 & 14 & 0 & 20 & 5 & 28 & 10 \\ 17 & 8 & 4 & 20 & 0 & 21 & 4 & 9 \\ 9 & 11 & 10 & 5 & 21 & 0 & 2 & 3 \\ 13 & 3 & 1 & 28 & 4 & 2 & 0 & 2 \\ 7 & 5 & 16 & 10 & 9 & 3 & 2 & 0 \end{pmatrix}$$

이 자선 단체는 도시락을 식당을 출발한 지 20분 내에 배송하고자 하는데 가능한 지 파악하여라.

문제 5. 식사 배달 - 답안

```
from ortools.linear_solver import pywraplp
import pandas as pd
```

```
DIST = [
    [0,10,12,5,17,9,13,7, 0],
    [10,0,9,20,8,11,3,5, 0],
    [12,9,0,14,4,10,1,16, 0],
    [5,20,14,0,20,5,28,10, 0],
    [17,8,4,20,0,21,4,9, 0],
    [9,11,10,5,21,0,2,3, 0],
    [13,3,1,28,4,2,0,2, 0],
    [7,5,16,10,9,3,2,0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0]
]
```

```
nCity = len(DIST)
```

```
solver = pywraplp.Solver.CreateSolver("SAT")
```

```
X = {}
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            X[i, j] = solver.IntVar(0, 1, "X"+str(i)+str(j))
```

```
Y = {}
for i in range(nCity):
    Y[i] = solver.IntVar(0, 1, "U"+str(i))
```

```
U = {}
for i in range(1, nCity):
    U[i] = solver.NumVar(1, nCity-1, "U"+str(i))
```

```
# 제약조건
# 도시 j로 한 번은 들어와야 한다.
for j in range(nCity):
    solver.Add(solver.Sum([X[i, j] for i in range(nCity) if i != j]) == 1, 'in_'+str(j))
```

```
# 도시 j로 부터 한 번은 나와야 한다.
for i in range(nCity):
    solver.Add(solver.Sum([X[i, j] for j in range(nCity) if i != j]) == 1, 'out_'+str(i))
```

```
for i in range(1, nCity):
    for j in range(1, nCity):
        if i != j:
            solver.Add(U[i] - U[j] + nCity*X[i,j] - (nCity-1)<= 0, 'U_'+str(i)+str(j))
```

```
# Objective
objective_terms = []
for i in range(nCity):
    for j in range(nCity):
        if i != j:
            objective_terms.append(DIST[i][j] * X[i, j])
```

```
solver.Minimize(solver.Sum(objective_terms))
```

```
if 1:
    with open('hw4-1.lp', "w") as out_f:
        lp_text = solver.ExportModelAsLpFormat(False)
        out_f.write(lp_text)
```

```
# Solve
status = solver.Solve()
```

```
# Print solution. 답안 출력
```